

Soft Decision Tree for Classification and Mitigating Vanishing Gradient using Truncated and Scaled Routing

Will Degroot, Xinyu Ni
Dartmouth College

Hard Decision Trees are "greedy", How to fix?

Traditional hard decision trees build tree top-down using greedy splits.

- Each node chooses locally optimal threshold
- Earlier splits cannot be revised
- Output non-continuous, constant approximation

This produces a piecewise model trained by sequential local optimization.

Soft Decision Tree

Replaces hard routing with probabilistic routing.

Each internal node computes a gating probability

$$g_m(x) = \sigma(w_m^T x + b_m), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

and returns a weighted mixture of its children until leaf node

$$F_m(x) = g_m(x)F_m^L(x) + (1 - g_m(x))F_m^R(x)$$

The probability of reaching leaf l becomes

$$\mu_\ell(x) = \prod_{m \in P(\ell)^L} g_m(x) \prod_{m \in P(\ell)^R} (1 - g_m(x))$$

The tree predicts

$$F_{root}(x) = \sum_{\ell} \mu_\ell(x) z_\ell$$

With training objective:

$$L = -[r \log y + (1 - r) \log(1 - y)]$$

Where $y = \sigma(F_{root}(x))$

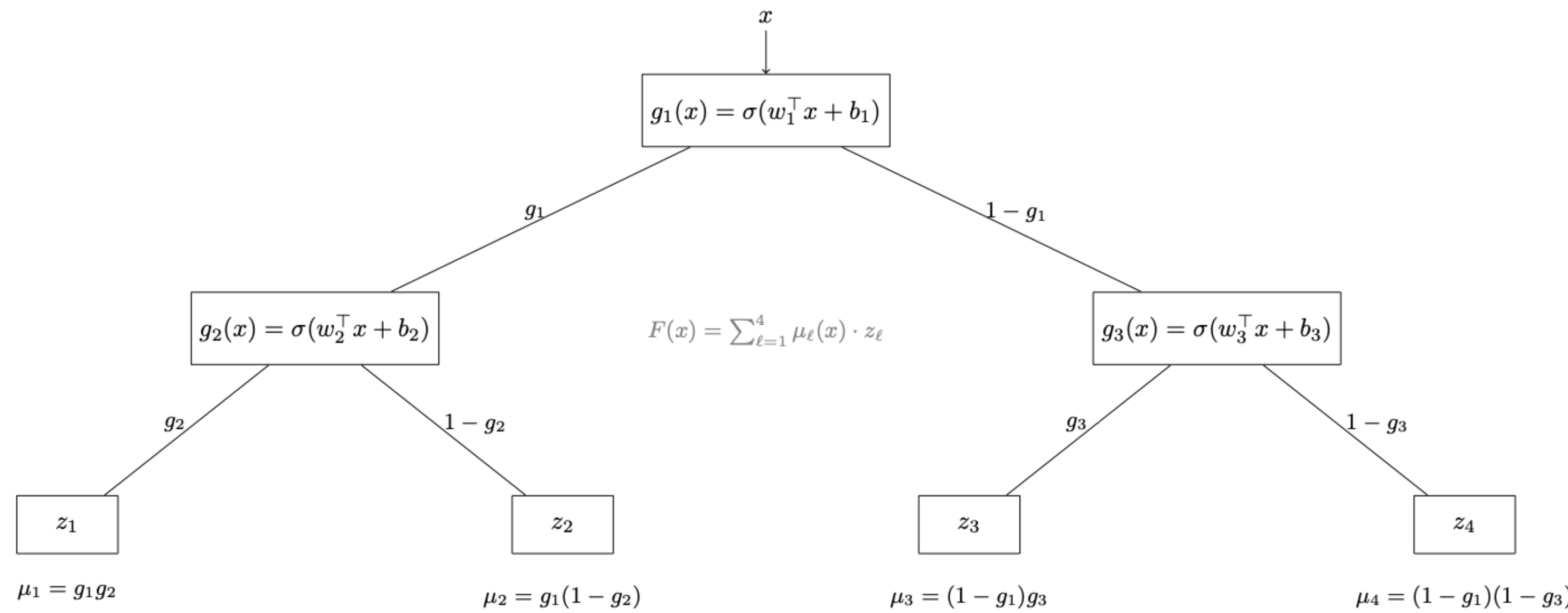


Figure 1. Ordinary Soft Decision Tree

Gradient Descent Training

Optimize with respect to parameters

$$\theta = \{w_m, b_m, z_\ell\}$$

Gradient at node m can be written as

$$\frac{\partial L}{\partial w_m} = \delta_m (F_m^L - F_m^R) g_m (1 - g_m) x$$

Where error signal at m is

$$\delta_m = \delta_{root} \mu_m(x)$$

Condensed Algorithm

1. Start by Initializing $w_m, z_m^L, z_m^R \sim \mathcal{U}(-0.01, 0.01)$
2. For each training sample, forward pass and compute root error:

$$\delta = \sigma(F_{root}(x)) - r$$

3. Propagate error to node m
4. Compute split gradient

$$\beta = \delta (F_m^L(x) - F_m^R(x))$$

5. Update the parameters until convergence

$$w_m \leftarrow w_m - \eta \beta g_m(x) (1 - g_m(x)) x$$

$$z_m^L \leftarrow z_m^L - \eta \delta g_m(x)$$

$$z_m^R \leftarrow z_m^R - \eta \delta (1 - g_m(x))$$

6. If validation error decreases, expand node m and train children recursively; otherwise keep m as a leaf

The Vanishing Gradient Problem

The gradient at node m contains routing probability

$$\mu_m(x) = \prod_{k \in \mathcal{P}(m)} \alpha_k(x)$$

Where $\alpha_k(x) \in (0,1)$ is the routing probability at ancestor node k. As the depth of tree increases, this term becomes a product of many probabilities, which shrinks exponentially, causing error signal from the leaf to disappear before updating the weights of earlier nodes.

$$\left\| \frac{\partial L}{\partial w_m} \right\| \propto \mu_m(x)$$

Additionally, the sigmoid derivative $g_m(x)(1 - g_m(x))$ attenuates the update when gates become confident. These effects cause deep nodes to receive weak updates, effectively freezing them during training.

Proposed Fixes to VG Problem

To mitigate the vanishing gradient problem, we introduce a localized training scheme.

When performing gradient descent on a node m within the tree, we instead descend with respect to the cross-entropy loss but with $y = \sigma(F_{root'}(x))$

Where $root'$ is the nth-parent of m (i.e., if n=2, then $root' = n.parent.parent$). Taking partials, we obtain gradients for $\{w_m, b_m, z_\ell\}$ which are the same functional form as before but with error signal $\delta_{m'} = (\sigma(F_{root'}) - r) \mu_{m \rightarrow root'}(x)$. We call this the localized optimization approach. Under this approach, the gradient no longer decay exponentially as the depth increases.

We also proposed a mixed error function/signal

$$L_{mix} = \lambda L_{global} + (1 - \lambda) L_{local} \text{ with } \lambda \in [0,1]$$

$$\delta_m^{mix} = \lambda \delta_m + (1 - \lambda) \delta_{m'}$$

Gradients for other parameters follow the same form.

Another approach to mitigating the vanishing gradient problem is to scale the gradient at node m as a function of its depth. If we assume that the mean gating probability throughout a tree is $\sim .5$, then the expected path probability decays roughly at $(.5)^d$ for nodes at depth d. Under this assumption, scaling the gradient by $2^{\text{depth}(m)}$ approximately compensates for this exponential attenuation. In practice, routing probabilities vary across nodes and samples and as a result, depth-based rescaling helps but is still a crude fix.

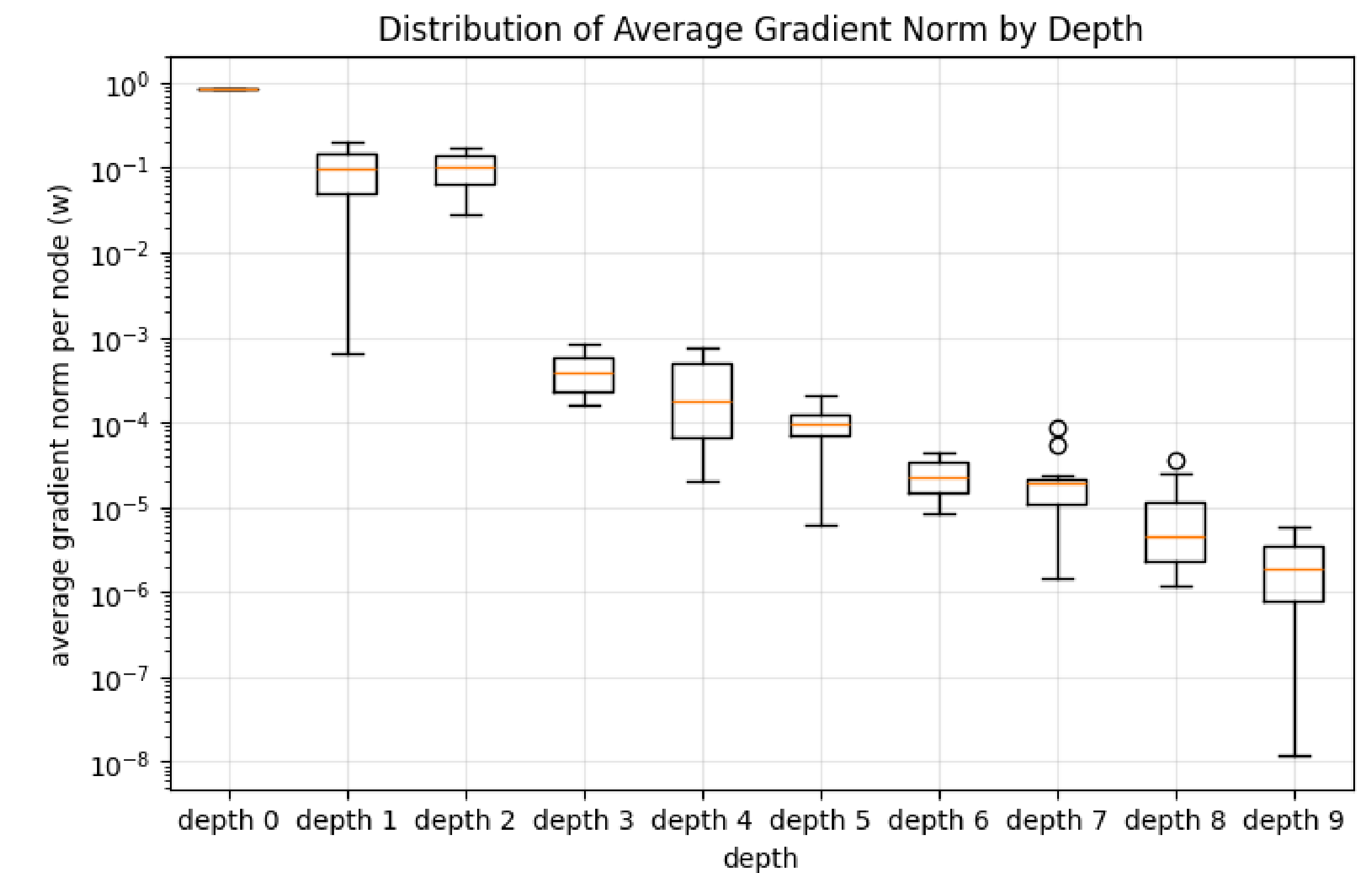


Figure 2. Gradient norm decays exponentially with Tree Depth

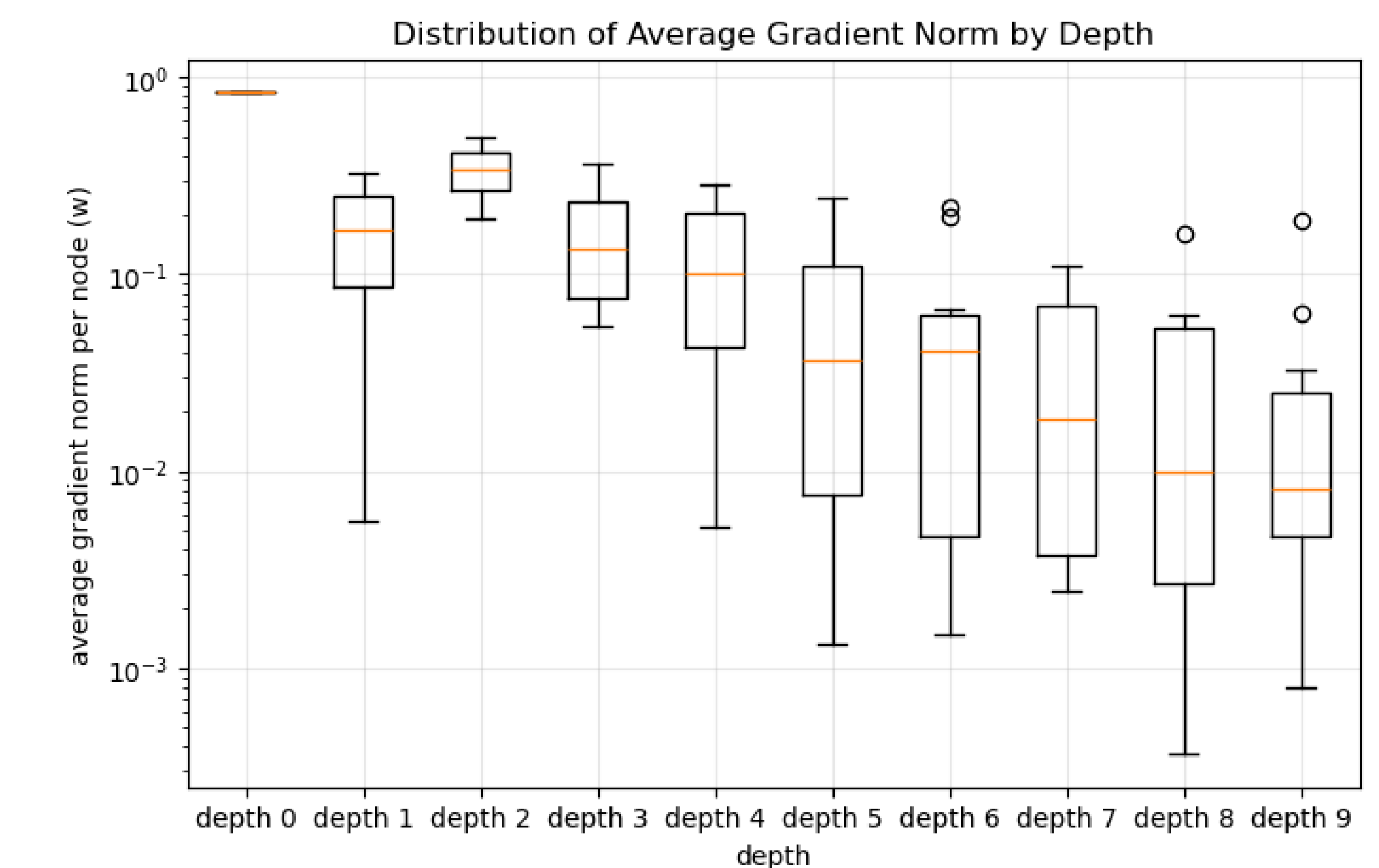


Figure 3. Modified Gradient vs. Depth

Model	Accuracy	Precision	Recall	F1
Logistic Regression	.8361	.7963	.4480	.5736
Ordinary Soft Decision Tree	.9327	.8615	.8655	.8635
Scaled Soft Decision Tree	.9410	.8824	.8772	.8798

Table 1. Classification Performance on Bike Share Dataset

Conclusion

Both of the localized gradient optimization approaches failed to yield a tree of depth $> \text{mod_depth} + 1$. We believe that this occurred because once $\text{depth}(m) > \text{mod_depth} + 1$, the local error function becomes distinct from the global error function. It is likely that the local error function was simply never able to understand enough global behavior to improve global error (in the datasets we trained on). Even in the mixed approach, setting $\lambda = .8$ or $.9$ was not enough to force the tree to grow beyond mod_depth . And since the local descent approach is the exact same as the ordinary descent approach whenever the depth of the tree is less than or equal to mod_depth , this means we never observed any advantage to using the local strategy. On the other hand, as one can observe, our scaled approach concretely improved metrics in comparison to the ordinary model.